DSA

ASSINGMENT

Submitted by :- Bhavin S

Submitted TO :- Amrendra Nath Tripathi

SAP ID :- 590022508

Batch :- 9

Bhavin S
590022508
Batch 9

SET-1 SECTION B

Q1.

* Static Memory Allocation

→ • Memory is allocated at compile time.
• Size of memory is fixed and cannot be changed later.
• Memory is usually allocated in stack or data segment.
• Allocation and deallocation are handled automatically by the compiler.
• Faster because allocation happens before the program runs

* ~~Fas~~ Dynamic Memory Allocation

• Memory is allocated at run time.
• Size of memory can be changed during execution.
• Memory is allocated in the heap.
• Programmer must manually allocated and free memory
• slightly slower due to runtime allocation.

Examples:-

• static memory Allocation

```
int arr[5]
```

• Dynamic memory Allocation

```
int *arr;
arr = (int*) malloc (5* sizeof (int));
```

Q2

→ * Abstract Data Type (ADT)

- An Abstract Data Type (ADT) is a logical model of a data structure that defines what operations can be performed on the data, without describing how those operations are implemented.
- In other words, ADT focuses on the behavior of the data structure rather than its internal implementation..
- eg, a stack ADT defines operations such as:
  - push() - insert an element
  - pop() - remove the top element
  - peek() - view the top element.
- But the ADT does not specify whether the stack is implemented using arrays or linked lists..
- Example:
  Stack, Queue, List, and Set are common examples of ADTs.

- Importance of ADT in Software Design

1. Abstraction - It hides the internal implementation details and shows only the required operations.
2. Modularity - Programs can be divided into smaller independent parts, making them easier to manage.
3. Flexibility - The internal implementation can be changed without affecting the rest of the program.
4. Code Reusability - The same ADT can be used in different programs or applications.
5. Better Program Design - Helps developers focus on problem solving rather than implementation details.

(Q3)

→ * Function Pointer as a Member of a Structure in C

- In C, a function pointer can be stored as a member of a structure. This allows a structure to call different functions using the pointer, making the program more flexible.
- A function pointer inside a structure stores the address of a function, and that function can later be called using the structure variable.
- This concept is commonly used when different operations need to be performed on the same type of data

Basic Syntax

```
Struct structureName {
    return_type (*functionPointerName)(parameters);
};
```

(Q4)

→ * Row-Major and Column-Major of a 2D Array:

- A 2D array is stored in computer memory as a linear sequence of elements. there are two common ways to store the elements in memory.

1. Row-Major order

- in row-major order, elements of the array are stored row by row.
- this means all elements of the first row are stored first, then the elements of the second row, and so on.

eg:- matrix A[2][3]

$$A = \begin{vmatrix} a11 & a12 & a13 \\ a21 & a22 & a23 \end{vmatrix}$$

storage order:-

a11 → a12 → a13 → a21 → a22 → a23

2. Column-Major order

In column-major order, elements are stored column by column.

All elements of the first column are stored First, then the second column, and soon.

~~Usin~~ Matrix A[2][3]

$$A = \begin{vmatrix} a11 & a12 & a13 \\ a12 & a22 & a23 \end{vmatrix}$$

Storage order :-

a11 → a21 → a12 → a22 → a13 → a23

(not used in C)

* Address Calculation in Row-Major Order

Suppose:

- Base address = B
- Size of each element = S
- Array dimensions = m×n
- Element = A[i][j]
- indexing starts from 0

Before reaching element A[i][j], the number of elements stored is:

i x n x j

Because:

- There are i complete rows before row i
  - Each row contains n elements
    - Then j elements inside the row

So total elements before A[i][j]:

i x n + j

Each element occupies S bytes, therefore the offset is:

(i x n + j) x S

Address. formula

The address of element A[i][j] in row-major order is:

Address (A[i][j]) = B + S x (i x n + j)

B = base address of the array

S = size of each element.

n = number of columns

i, j = row and column index

(QS.)

Iterative vs Recursive Binary Search

Binary Search is an efficient searching technique used to find an element in a sorted array. It works by repeatedly ~~dividly~~ dividing the search interval into two halves.

1. Iterative Binary Search

- In the iterative approach, a loop is used to repeatedly divide the array until the element is found or the search space becomes empty.

- ~~Pes~~ Pseudocode (Iterative)

> Binary Search Iterative (A, n, key)

low = 0

high = n-1

While low ≤ high

mid = (low + high) /2

if A [mid] == key

return mid

else if Key < A [mid]

high = mid - 1

else

low = mid + 1

return -1

2. Recursive Binary Search

In the recursive approach, the function calls itself to search in the left or right half of the array.

Pseudocode (Recursive)

```
Binary Search Recursive (A, low, high, key)
if low > high
        return -1
mid = (low+high)/2
if A (mid) == key
        return mid
else if key < A [mid]
        return Binary Search Recursive (A, low, mid -1, key)
else
        return Binary Search Recursive (A, mid +1, high, key)
```

* Time Complexity

Both iterative and recursive versions reduce the search space by half each step.

Best case:

O(1) (element found at middle)

Average case:

O(logn)

Worst case:

O(log n)

* When to Prefer Each

> Iterative Binary Search is preferred when:

- Memory usage must be minimal
- Faster execution is required
- Avoiding recursion overhead

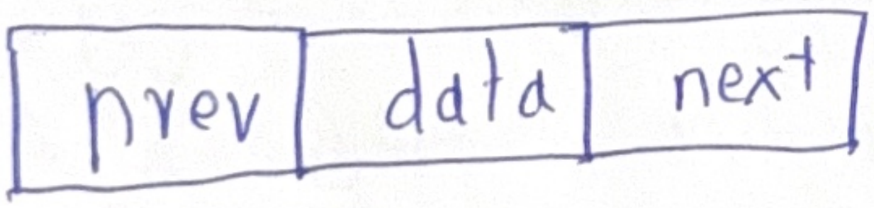
> Recursive Binary Search is preferred when:

- Code readability and simplicity are important
- The algorithm naturally fits recursive logic

Q6 → Doubly linked List Operations

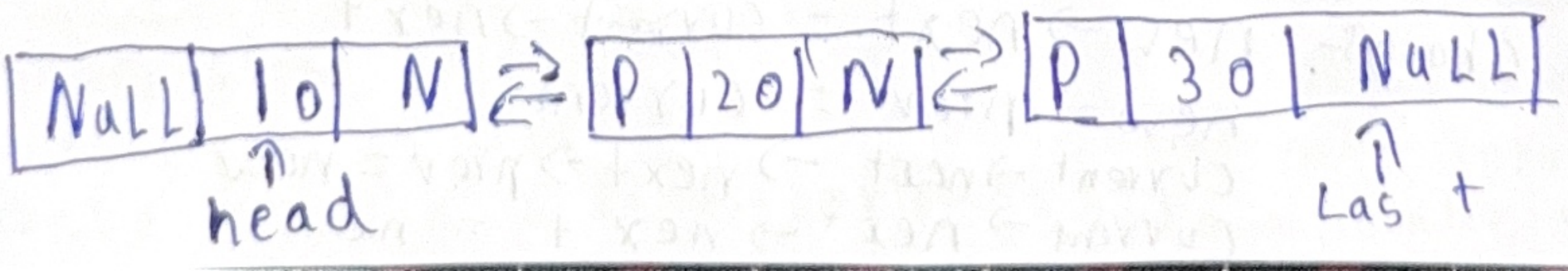
- A Doubly linked List is a linked list where each node contains three parts:

1. Pointer to previous node (prev)
2. Data
3. Pointer to next node (next)

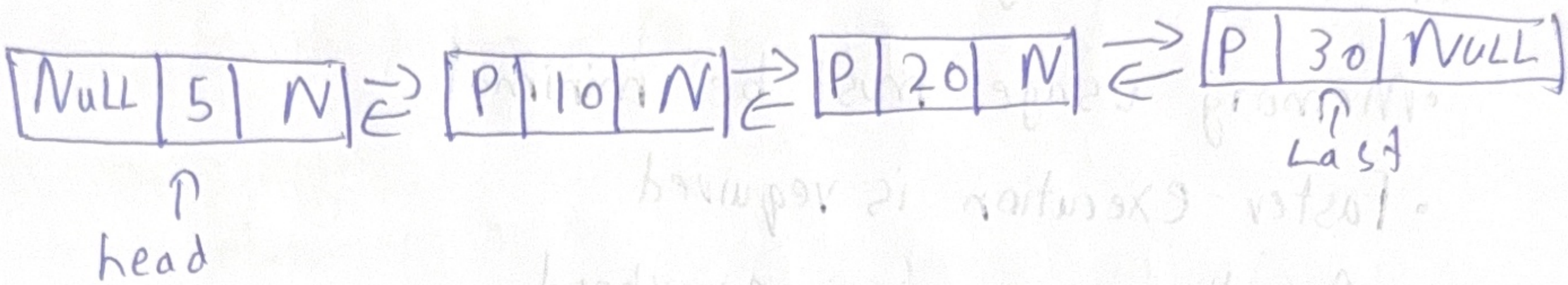


Each node is connect in both directions, so traversal is possible forward and backward.

Sample Doubly linked list



1. Insertion at Beginning (5)



Pointer changes:-

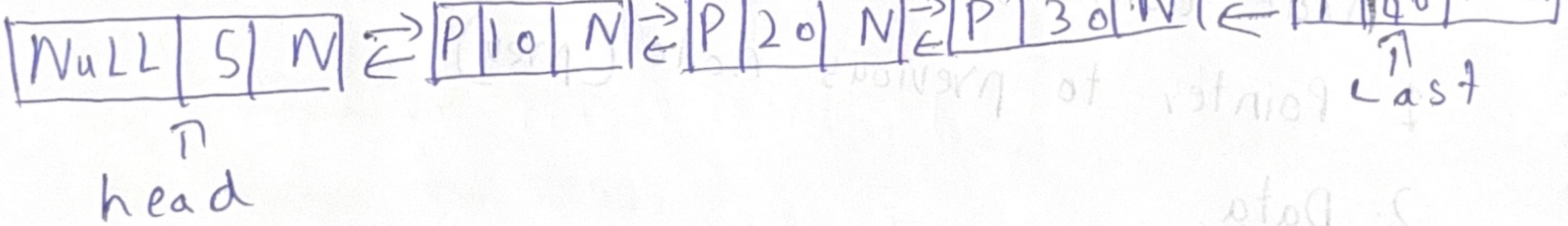
new → next = head

new → prev = NULL

head → prev = new

head = new

2. Insertion at End (40)



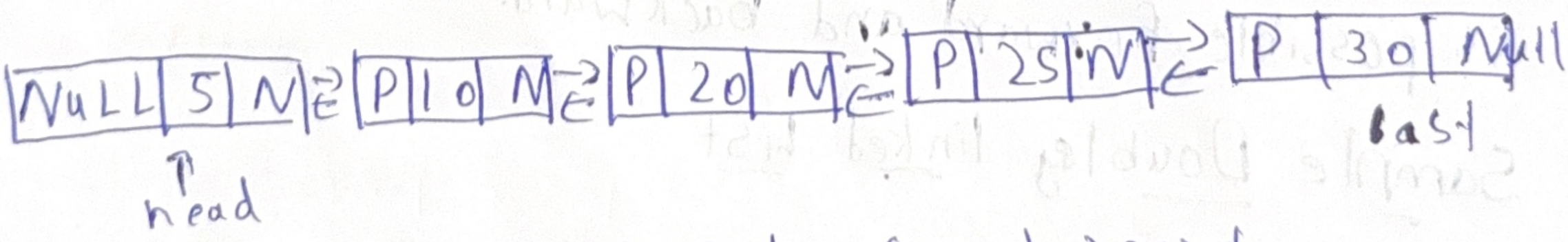
Pointer changes:-

new → prev = last

new → next = NULL

last → next = new

3. Insertion in Middle (25)



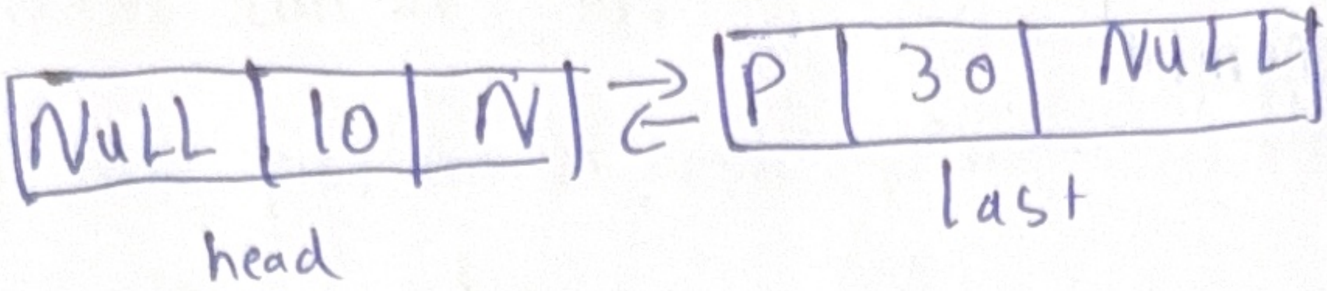
Pointer changes- new → next = current → next

new → prev = current

current → next → next → prev = new

current → next → next = new

4. Deletion of a Node (20)



pointer changes:-

```
temp → prev → next = temp → next
temp → next → prev = temp → prev
free (temp)
```

(Q7) Generalized linked List (GLL)

- A Generalized linked list is a linked list in which a node can store either a data element or a pointer to another linked list.

This allows representation of nested or hierarchical data structures.

A typical node contains:-

tag | data / down | next

- tag → indicates whether the node contains data or a Sublist
- data / down → actual value or pointer to another list
- next → pointer to the next node.

Difference from standard Linked List

Standard Linked List	Generalized linked list
• Nodes store only data	• Nodes store data or sublists
• Structure is linear	• Structure can be nested
• Node contains data and next pointer	• Node may contain tag, data, down pointer, and next pointer

* Example: Polynomial Representation

- Consider the polynomial:

$3x^2y + 2xy^2 - 5$

- Each term contains a coefficient and powers of variables.

Representation Idea:

Node 1 → coefficient 3, variables x^2, y^1

Node 2 → coefficient 2, variables x^1, y^2

Node 3 → coefficient -5

The variable powers for each term can be stored as a sublist attached to the main node, which is why a generalized linked list is suitable for representing multivariable polynomials.

(8a)

a) Amortized Analysis

Amortized analysis determines the average cost per operation over a sequence of operations, even if some operations are expensive.

1. Aggregate Method

Total cost of n operations is calculated and then divided by n to find the amortized cost per operation

eg:-

- In a stack withe operations push, pop, and multipop, each element can be popped only once.
- If we perform n push operations, the total number of pops cannot exceed n.

Total cost $\leq 2n$

Amovtized cost per operation = $2n/n = O(1)$

2. Accounting Method

- In this method, we assign an artifical cost (charge) to operations
- Some operations ave charged extra, and the extra amount is saved to pay fov expensive opevations later.

eg:-

- stack operations:-
  - push → charge 2units
  - pop → charge 0units
- 1 unit pevforms the push, and the extva unit is stoved to pay for the pop later. Thus the amovtized cost-pev opevation remains O(1).

3. Potential Method

- This method uses a potential function Φ to measuve the stoved energy in a data structure.
- Formula

  Amovtized cost = Actual cost + Φ(new) - Φ(old)

- Eg:-
- Dynamic avvay vesizing

  Although vesizing costs O(n) occasionally, the amovtized insevtion cost is O(1).

b) Asymptotic Notations

- Asymptotic notations describe the growth rate of algorithms as input size increases.

1. Big O Notation (O)

- Big O gives the upper bound of a function.

$f(n) \leq c \cdot g(n)$ for $n \geq n_0$

- It represents the worst-case growth rate.
- eg:- Binary search → $O(\log n)$

2. Omega Notation (Ω)

- omega gives the lower bound of a function.

$f(n) \geq c \cdot g(n)$ for $n \geq n_0$

- It represents the best-case growth rate.
- eg:- linear search best case → $\Omega(1)$

3. Theta Notation (Θ)

- Theta gives the tight bound (both upper and lower).

$c_1 g(n) \leq f(n) \leq c_2 g(n)$

- eg:- Merge sort → $\Theta(n \log n)$

* Proof that

- $f(n) = 3n^2 + 5n + 2$ is $O(n^2)$
- $f(n) \leq c \cdot n^2$

> for $n \geq 1$

- $3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$

> Thus

$f(n) \leq 10n^2$

> choose

$c = 10$

$n_0 = 1$

$\therefore f(n) = O(n^2)$

(9Q) Array operations on an Unsorted Array

a) Insertion at a Specified Position

Algorithm

1. Start
2. Insertion (A, n, pos, element)
3. If pos < 1 or pos > n+1
   print "Invalid position"
4. for i = n down to pos
   A[i+1] = A[i]
5. :A[pos] = element
6. n = n+1
7. Stop.

- All the elements from the specified position are shifted one step to the right, then the new element is inserted.

b) Deletion of a Specified Element (first occurrence)

Algorithm

1. Start
2. Deletion (A, n, key)
3. For i = 0 to n-1
4. If A[i] == key

5. For j = i to n-1
6. A[j] = A[j+1]
7. n = n-1
8. Stop
9. IF element not found
print "Element not found"

C) Searching

- Linear Search

Algorithm

1. Start
2. Linear Search (A, n, key)
3. Set i = 0
4. while i < n
5. if A[i] == key
6. print "Element found at position", i
7. stop
8. i = i+1
9. print "Element not found"
10. Stop

- Each element is checked one by one until the key is found or the array ends.

- Binary Search

Algorithm

```
1. Start
2  Binary search (A, n, key)
3.  low = 0
4.  high = n-1
5.   while low ≤ high
6.            mid = (low + high) / 2
7.          If A [mid] == key
8.                  return mid
9.          If key < A [mid]
                  high = mid - 1
10.         Else
11.               low = mid+1
12.  return -1
```

- The array is repeatedly divided into two halves, reducing the search space each step.

* Complexity Comparison

Search Method	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

100 Skip List

- A skip list is a probabilistic data structure that allows fast search, insertion, and deletion. It consists of multiple levels of linked lists, where higher levels act as shortcuts to skip over elements.
- The bottom level contain all elements, while upper levels contain fewer elements.

a) Structure of skip list

Example keys: {2, 5, 8, 12, 17, 21}

Skip list with 3 levels

level 3: HEAD → 2 ⟶ 12 ⟶ 21 → NULL

level 2: HEAD → 2 ⟶ 8 ⟶ 12 ⟶ 17 ⟶ 21 → NULL

level 1: HEAD → 2 ⟶ 5 ⟶ 8 ⟶ 12 ⟶ 17 ⟶ 21 → NULL

- level 1 → contains all elements
- Higher levels → contain selected elements that provide shortcuts

Probabilistic Promotion

When a new element is inserted:-

1. It is first inserted at level 1.
2. A coin flip (probability $p = 0.5$) decides whether the node moves to the next level.
3. If heads appears, the node is promoted to the next level.
4. This continues until a tails appears or the maximum level is reached

This random promotion keeps the structure balanced on average.

b) Search Algorithm

Algorithm

1. Start
2. Skip search (head, key)
3. Start at the highest level
4. while current node exists
5. IF next node key ≤ search key
6. move right
7. Else
8. move down one level
9. IF Key Found
10. return position
11. Else return not found
12. Stop.

- search begins at the top level.
- move right while keys are smaller
  - when the next key becomes larger, move down one level.
  - Continue until the key is found or the bottom level is reached.

* Time complexity
  - Expected search time:

    O (log n)

    Because higher levels allow skipping many elements.
  - worst case (rare):

    O (n)

* Comparison of Data structures

Operation	Singly linked list	Skip List	Balanced BST
Search	O(n)	O(logn)	O(logn)
Insertion	O(n)	O(logn)	O(logn)
Deletion	O(n)	O(logn)	O(logn)